TwinCatAdsTool, a manual

This tool reads the **PERSISTENT** variables out of a Beckhoff TwinCAT controller into a file, writes them back, and compares the two. It also watches live values and plots them.

The reason it exists is that persistent variables are the part of a machine that is **not** in the PLC project. Calibration measured on site, recipes, homing positions, counters, whatever was tuned by hand during commissioning: none of that is in source control, and none of it survives a controller swap, a change to the layout of a persistent structure, or a firmware upgrade. The program comes back in five minutes. Those values do not come back at all unless somebody has a copy.

Contents

1. Before you start
2. Connecting
3. Taking a backup
4. Restoring
5. Proving a restore worked
6. Comparing
7. Watching values
8. The scope
9. Watch sets
10. From the command line
11. Where the files are
12. When something goes wrong
13. What is verified, and what is not

---

1. Before you start

On the machine running this tool you need the TwinCAT ADS runtime installed. That is what provides the router every ADS message goes through. TwinCAT 4024 and 4026 both work, and the same executable serves them.

An ADS route to the controller must exist. A route is a permanent, named, authenticated path between two ADS machines. It is not the same as being able to ping the controller: you can have a perfect network connection and no route, and then nothing works.

Routes are made in the TwinCAT system tray icon, under *Router, Edit Routes*, or from the controller side. You need the controller's **AMS net id**, which is six numbers separated by dots such as `5.24.108.31.1.1`, and normally its password.

The controller needs a PLC runtime that is started. A controller in Config mode publishes no PLC symbols, and this tool has nothing to read.

What counts as persistent

Only variables declared `PERSISTENT` are read and written. In Structured Text that is

```
VAR_GLOBAL PERSISTENT
    rCalibration : REAL;
    stRecipe     : ST_Recipe;
END_VAR
```

A structure declared persistent makes **every member of it persistent in its own right**, which is why a backup of one `STRUCT PERSISTENT` shows up as an object with all of its members inside.

`RETAIN` variables are a different mechanism and are not covered.

---

2. Connecting

The bar across the top of the window is the connection, and it applies to every tab.

1. **Device** lists the controllers found by an ADS broadcast on the local network. Pick yours if it appears. The search only finds machines that answer a broadcast, so a controller behind a router will not be in the list even when it is perfectly reachable.
2. **Address** is the AMS net id. You can type it in directly, which is what you do when the broadcast did not find the controller.
3. **Port** is the PLC runtime. `851` is the first PLC task in TwinCAT 3 and is almost always the right answer. `801` is TwinCAT 2.
4. **Connect**.

When the state shows *Connected* together with the controller's ADS state (`Run`, `Stop`, `Config`), the tabs on the left become usable.

A controller in **Run** can be backed up and restored. Whether restoring onto a running machine is wise is a question about your plant, not about the tool: the values change under you while they are being written, and the program may overwrite what you just wrote. For anything you intend to verify afterwards, stop the PLC first.

---

3. Taking a backup

Backup tab, then **Read from PLC**.

Every persistent variable is read and shown as JSON. Then **Save to file** writes it where you say.

A backup file looks like this:

```
{
  "MAIN": {
    "nCycleCount": 18422,
    "stAxis": {
      "fHomeOffset": 12.75,
      "bCalibrated": true
    }
  },
  "GVL": {
    "aRecipe": [ 10, 20, 30, 40 ]
  }
}
```

It is ordinary JSON, on purpose. You can read it, edit it in a text editor, put it in version control, diff two of them with any tool you like, or generate one from a script.

Read the report

Report shows what happened to every variable, and it is worth reading rather than assuming.

```
71 ok, 0 failed, 0 skipped in 0.1 s
```

- **ok** means the variable is in the file.
- **failed** means it could not be read, with the ADS error that says why.
- **skipped** means it was left out.

A backup that says anything other than `0 failed, 0 skipped` **is not a full backup**, and the file will be missing exactly the variables named in the report. Save it if you want, but write down what is missing from it.

How often

A backup takes a fraction of a second, so there is no reason to be sparing. Sensible moments:

- at the end of commissioning, when the machine is right
- before any download that changes the declaration of a persistent variable
- before a TwinCAT or firmware upgrade
- before swapping a controller
- on a schedule, which is what the command line is for

---

4. Restoring

Restore tab, then **Load from file**. The variables from the file are listed. **Write to PLC** asks for confirmation, then writes.

Restoring writes each individual value to the symbol that owns it. That matters for a reason worth knowing: the ADS library hands out a **copy** of the buffer when you step into a structure member or an array element, so the older approach of reading a variable, changing it in memory and writing it back could not reach anything below the first level, and reported success anyway. Writing every leaf to its own symbol takes that copy out of the path.

Read the report, again

```
49 ok, 0 failed, 0 skipped in 1.9 s
```

- **failed**: the value was refused, with the ADS error.
- **skipped**: the variable is on the PLC but not in the file, so it was **left alone**. Skipped is not an error, but it does mean the plant now holds a mixture of the file and whatever was there before.

Variables in the file that no longer exist on the PLC are reported too. That is the usual sign of a file taken from a different version of the program.

What can go wrong at the value level

The report names the variable and the path within it, for example:

```
MAIN.stAxis.fHomeOffset: backup value 'abc' does not fit the plc type
```

This is what you get when a file has been hand edited into something the PLC type cannot hold. The rest of the restore still runs; only that leaf is refused.

---

5. Proving a restore worked

A report saying success is not proof that the plant holds what the file says. That is not a theoretical worry: it is the defect this fork was written to fix, and it went unnoticed for a long time precisely because the report looked fine.

The way to be sure is to read the plant back and compare it against the file:

1. Restore the file.
2. Take a fresh backup into a second file.
3. Compare the two.

The Compare tab does this by eye, and `compare` on the command line does it by exit code.

Do this with the PLC stopped. On a running machine the program writes its own values while you are comparing, and there is no way to tell a value the restore failed to write from one the program has changed since. That distinction cost a wrong conclusion once already.

---

6. Comparing

Compare tab. Each side can be filled either from the PLC or from a file, so you can compare

- a file against the plant, which is the verification above
- two files, for instance last month's backup against today's
- the plant against a file taken from another machine of the same type

The comparison is made **value by value**, not as text. Every row is one symbol on the PLC, named by its path, with the reading each side holds. The count at the top says how many values differ, and a coloured stripe on each row says how:

orange	both sides have this value and they disagree
red	only the left side has it
green	only the right side has it

A value that exists on one side only usually means the PLC program has changed since the backup was taken: a variable was added, removed or renamed.

Finding the differences

Only differences is on to begin with, so the list holds nothing but what changed. Turn it off and every value of the backup is listed — which is how you confirm that a value has *not* moved, rather than only seeing the ones that have.

The four chevrons move between differences: first, previous, next, last. With the filter off they skip past everything the two sides agree on.

Correcting the PLC from the other side

Where a value differs and both sides have it, the two arrows in the middle of the row offer to carry it across. Clicking an arrow **picks** the value; it does not write it. Clicking the same arrow again takes the pick back.

- **All to left** / **All to right** picks every difference at once
- **Undo all** drops every pick
- **Write to plc** is the only thing that touches the machine, and it asks first

The top of the tab says how many values are picked and waiting. Nothing is sent until you press *Write to plc* and confirm.

Only the PLC is written to. A backup file is left exactly as it was found. So the direction that is open depends on which side you filled from the PLC: read the plant into the left and values travel left, into the right and they travel right. If neither side came from the PLC, the arrows are dead and a line under the count says so.

A value only one side has cannot be carried across, and no arrows are offered for it. Writing it would mean creating a variable on the PLC, and ADS writes values — it does not declare symbols.

After the write the PLC side is read back automatically, so what is on screen is what is on the machine, including anything the write could not place.

What it writes, and what it leaves alone

Exactly the values you picked. The variables you did not touch are not read, not written and not reported: a merge of three values says three values, not "eleven thousand skipped".

The write goes through the same machinery as a restore, so a value that cannot be placed — the PLC declares it read only, the type no longer fits, the variable no longer exists — is reported by name rather than silently dropped.

---

7. Watching values

Explore tab. This is where you look at live values and change them.

The left panel is the symbol tree. **Search** finds a symbol by any part of its path, which is usually faster than opening the tree. The circular arrow re-reads the symbol list from the PLC, which you need after downloading a new program.

Every symbol in the tree, and every result of a search, carries a **+** on its right — the tooltip says *Watch this symbol*. Pressing it adds that symbol to the **watch list**, the table filling the top right of the tab. Nothing is watched until you press it, and the list is the only place a live value is ever shown.

Only leaves carry the **+**: a structure or an array has no single value to show, so open it and take the member you want.

The watch list shows, for each symbol, from left to right:

Column	
✕	drop the symbol from the list, which also takes it off the plot
Name	hover for the full path and the comment from the declaration
Type	the PLC type as declared
P	the variable is persistent, so it is one of those the Backup and Restore tabs act on
Value	live, updated from the PLC
New value	type a value here and press the send button on its right to write it
chart buttons	put the signal on the scope, or take it off — see below

The editor offered under *New value* follows the type: a switch for a `BOOL` or a `BIT`, a number box for numeric types, a time picker for `TIME` and `DATE_AND_TIME`, a text box for strings.

Writing a value here writes it to the running machine immediately. There is no confirmation.

---

8. The scope

The lower half of the Explore tab is the plot. It is empty until you put something on it, and what you can put on it are the symbols already in the watch list.

Putting a variable on the scope

It takes two steps, and the second one is the one people look for in the wrong place: the button that plots a signal is **not** in the symbol tree, it is at the far right of the row in the watch list.

1. In the tree or in **Search**, press the **+** beside the symbol. It appears in the watch list.
2. In its row in the watch list, scroll right to the last column and press the **chart** button (tooltip *Add Graph*). The signal starts being drawn immediately.

The **–** beside it (*Remove Graph*) takes it off the plot but leaves it in the watch list, still being read. The ✕ at the start of the row removes it from both.

If the row has no chart buttons at all, that type cannot be plotted. They are shown only for numeric types, `BOOL` and `BIT`. A `STRING`, a `TIME` or a `DATE_AND_TIME` can be watched and written, but there is nothing to draw, so the buttons are absent rather than disabled — nothing is broken and no setting will bring them back.

A `BIT` — the one-bit type a packed `STRUCT` or `UNION` is built from — is watched, plotted and triggered on exactly like a `BOOL`. Writing one is safe as well: it does not own the byte it sits in, so the value is written through the symbol, which sets that bit and leaves the other seven alone.

Repeat for as many signals as you want on the plot. `BOOL`s and `BIT`s go into lanes of their own along the bottom, the rest share the band above, each with its own scale.

Recording and viewing are two different spans

Memory	how far back the recording is kept
Window	how much of it is on screen

This separation is the whole point. Scrolling and zooming move the window **inside** the recording, so you can stop and go back to look at something that has already scrolled past.

The controls

Stop / **Start**	Stop freezes the view as well as the recording. Start returns to the live edge.
◀ ▶	move by a quarter of a window
🔍	halve or double the window
Live	appears once the view has left the present; takes you back to it
🧽	discard the recording
CSV	export the window being shown
PNG	save the plot as a picture
window icon	move the scope into a window of its own, as large as you like

The **mouse wheel zooms**, and **shift with the wheel scrolls**.

Digital signals

A `BOOL` or a `BIT` gets a lane of its own along the bottom and is drawn as steps. Analogue signals share the band above, each with its own scale. A bit does not travel through the values between 0 and 1, so drawing a transition as a slope would be a lie about the machine.

Sample ms

This is the **ADS notification cycle**: how often the controller looks at the symbol to see whether it changed. The default is 200 ms, which is also what this tool used to be fixed at without saying so. Lower it to see shorter events.

Two things this number does not promise:

- it is how often the server **looks**, not how often it transmits. The mode is still on change.
- the timestamps are taken when the notification **reaches the PC**. Going down to 10 ms makes the *order* of edges visible, not their *duration* to the millisecond. For faithful timing you need code inside the PLC, which is what TwinCAT Scope is for.

Trigger

The question actually asked of a scope on a machine is not what the last ten minutes looked like, but what happened around the moment something changed.

The trigger row sits above the plot, starting with the words **Trigger when**.

A variable can only be a trigger once it is on the plot. The first dropdown lists the plotted signals and nothing else: while the plot is empty it is empty too, and there is no way to type a symbol name into it. So a variable that is only being watched cannot be armed on — put it on the plot first, with the two steps above, and it appears in the list straight away.

Left to right along that row:

Control	
Trigger when ▾	which of the plotted signals to wait on
condition ▾	*goes TRUE*, *goes FALSE*, *rises above*, *falls below*
level	the number the last two are measured against; ignored by the first two
Arm	start waiting

Press **Arm** and the button becomes **Disarm**, with **Waiting** beside it. When the condition fires, that text becomes **Triggered at 14:32:07.881**.

You do not have to press **Start** first: arming restarts the recording and returns the view to the live edge on its own, because waiting for something to happen only makes sense from the present.

When it fires, the recording carries on for **half a window** so the aftermath is captured too, and only then does everything hold still with the event in the middle, marked by a line.

Disarm stops waiting. Taking the trigger signal off the plot also disarms, and clears the choice of signal — a trigger waiting on something no longer being drawn would wait for ever without saying why.

Every condition is a crossing and never a state. A signal already TRUE when you arm has not just gone TRUE, so it will not fire; otherwise arming would fire instantly every time and catch nothing. To catch a bit that is already up, wait for it to go FALSE, or arm before the machine gets there.

CSV export

The exported file covers the **window being shown**, not the whole recording, because the slice on screen was framed deliberately.

The layout is one column per signal and one row per instant at which any of them was read. Signals do not change together, so every cell carries the value its signal **was holding** at that instant, which is what the signal actually was and what makes the file plottable in a spreadsheet without further work.

The field separator follows your regional settings: where the decimal mark is a comma, a semicolon separates the fields.

---

9. Watch sets

Save set writes the symbols you are watching to a JSON file. **Load set** reads one back.

```
{
  "variables": [
    { "path": "MAIN.bEnable", "graph": true },
    { "path": "GVL.rSpeed",   "graph": true },
    { "path": "MAIN.sState",  "graph": false }
  ]
}
```

The format is deliberately flat and meant to be edited by hand. `graph` says whether the symbol also goes on the scope.

Values are not saved. A watch set says what to look at; carrying stale readings in the same file would invite reading them as measurements.

Paths the PLC does not have are collected and reported together with what ADS answered. A set written against another version of the program is exactly when knowing which symbols have gone matters.

---

10. From the command line

Everything the Backup and Restore tabs do can be asked for without opening the window.

```
TwinCatAdsTool backup  <netid> <port> <file>
TwinCatAdsTool restore <netid> <port> <file>
TwinCatAdsTool compare <netid> <port> <file>
TwinCatAdsTool --help
TwinCatAdsTool --version
```

Same executable, same engine as the window. With no arguments it opens the window as usual.

Exit codes

A script has no other way of learning how a run went, so the codes separate the cases a script would treat differently.

`0`	done, and every variable was processed
`1`	the command line could not be understood
`2`	the PLC could not be reached, worth retrying
`3`	the run finished, but variables failed or were skipped
`4`	something unexpected went wrong
`5`	`compare` only: the PLC and the file differ

`3` is the one that matters. An incomplete backup still writes what it managed to read, and a script that only checks whether the file exists would keep it as though it were whole.

Making errorlevel work

The application is a Windows program rather than a console one, so `cmd` moves straight to the next line while it is still running. `start /wait /b` is what makes the exit code readable. In PowerShell the equivalent is `Start-Process -Wait`. Neither is needed for a scheduled task or for `NT_StartProcess` called from the PLC.

A nightly backup

```
for /f "tokens=2 delims==" %%d in ('wmic os get localdatetime /value') do set t=%%d
start /wait /b TwinCatAdsTool.exe backup 5.24.108.31.1.1 851 D:\backups\plant_%t:~0,8%.json
if errorlevel 3 echo INCOMPLETE BACKUP >> D:\backups\alarm.txt
```

`if errorlevel 3` is true for 3 and anything above it, which is what you want here.

A restore that verifies itself

```
start /wait /b TwinCatAdsTool.exe restore 5.24.108.31.1.1 851 D:\backups\plant.json
start /wait /b TwinCatAdsTool.exe compare 5.24.108.31.1.1 851 D:\backups\plant.json
if errorlevel 1 echo THE PLC DOES NOT MATCH THE FILE
```

The second line is the one worth having.

Started by the PLC

`NT_StartProcess` from the TwinCAT PLC library will run it, which lets the machine take its own backup before it does something risky to itself.

Coming from Beckhoff's Symbol Explorer

`--SnapShotFromPlc`, `--SyncPlcToSnapShot` and `--SyncSnapShotToPlc` are accepted as spellings of `backup`, `backup` and `restore`, so an existing script can be pointed here without being rewritten.

---

11. Where the files are

Backups go wherever you saved them. The tool suggests `Backup_yyyy-MM-dd-HHmmss.json`.

The log goes to `logs\` beside the executable. If that folder cannot be written, which is normal when the executable sits somewhere protected, it goes to `%LOCALAPPDATA%\TwinCatAdsTool\logs` instead. The window tells you which on startup if there is a problem.

`log.config` beside the executable is optional. Without it the tool configures its own rolling log file. It is there for anyone who wants to change what is logged.

The theme you choose is remembered between runs.

---

12. When something goes wrong

`AdsException: Cannot register Port '0'`

The ADS 5.x client library against TwinCAT 4026. This fork uses ADS 7, which serves both 4024 and 4026, so you should not see it here. If you do, you are running an older build.

Connected, but no symbols

The PLC runtime is not started, or you are on the wrong port. Check the ADS state in the connection bar: `Config` means there is no running PLC to read.

The connection never comes up

Almost always the route rather than the network. Check *Router, Edit Routes* in the TwinCAT tray icon, and check that the local machine has an AMS net id of its own. A machine without one cannot open an ADS connection at all, and the error it gives does not say so.

A backup says `failed` or `skipped`

Read the report. It names each variable and the ADS error. A common cause is a symbol the PLC declares but will not hand over, for instance one that is write only.

A restore says `does not fit the plc type`

The value in the file cannot be converted into what the PLC declares. Usually a hand edited file, or a file from a version of the program where that variable had a different type.

The restore is slow

The log carries a breakdown:

```
Restore timing: scan 0.04 s, read 0.13 s, plan 1.32 s, prepare 0.00 s,
                transfer 0.39 s for 103234 values in 12 commands
```

`transfer` is set by the **number of commands** far more than by how many values they carry. If you see many commands, values are being split more than they need to be, which happens when a controller refuses a large one and the tool halves and retries. The retries are logged as well.

The scope shows nothing for a signal

Only numeric and boolean types can be plotted. Strings and times are watchable but not plottable, and their rows in the watch list have no chart buttons at all rather than greyed out ones.

The trigger dropdown is empty, or does not list the variable I want

It lists the signals **on the plot**, not the ones in the watch list. Put the variable on the plot first: the chart button at the right of its row in the watch list.

A watch set will not load some symbols

The message says what ADS answered for each. A path that exists in the tree but not in the flat symbol list, which is normal for anything below an array element, is resolved by asking the server directly, so this is usually a symbol that has genuinely gone from the program.

---

13. What is verified, and what is not

Verified on a real installation: connection on TwinCAT 4026 with ADS 7; a backup of 71 variables in 0.1 s; a restore of 10,978 leaves in which every leaf was given a distinct value, written, read back and compared one by one, with **10,978 out of 10,978 correct**; nested branches, arrays of structures, arrays inside structures inside arrays, strings, REALs, BOOLs, TIME, LTIME and DT. Backup and restore from the command line against a running controller.

Verified by test: 154 unit tests on every build.

Multidimensional arrays, `ARRAY[0..n,0..m]`, are covered too, on a second plant: an `ARRAY[0..30, 0..30] OF DUT_Cass`, **92,256 leaves in one variable**, restored with 92,256 correct and none out of place. The backup flattens such an array into a single list, and this is what says that the flattening and the write path agree on the order.

Not verified: the accuracy of scope timings below roughly ten milliseconds, and `compare` from the command line against a plant. Of the multidimensional case, only one shape has been exercised — square,

rank 2; rank 3 or a non square array would test the flattening further.